

3. 멀티턴 대화형 프롬프트 설계

1. 멀티턴 대화의 개념 소개 (싱글턴 vs 멀티턴)

싱글턴 대화는 한 번의 질문-응답으로 완료되는 상호작용을 말합니다. 사용자가 한 번 질문하거나 명령을 입력하면 AI가 그 입력만을 근거로 답변하고 대화가 끝나죠. 예를 들어, “오늘 서울 날씨 어때?” 라고 물으면 AI가 바로 날씨 정보를 답하고 대화를 종료합니다. 이러한 방식은 정보 조회처럼 간단하고 한 번에 답변이 가능한 작업에 효율적입니다.

반면, 멀티턴 대화는 한 번의 질의응답으로 끝나지 않고 여러 차례의 주고받는 대화로 이어지는 상호작용을 의미합니다. 사용자의 요청을 해결하기 위해 AI가 추가 질문을 하거나, 사용자가 답변을 듣고 후속 질문을 이어갈 수 있는 대화 흐름이 특징입니다. 예를 들어, 레스토랑 예약을 원하는 사용자가 있다면 처음에 “식당 예약하고 싶어요.”라고 요청한 후, AI는 “몇 분이 오시나요?”, “원하는 시간대가 언제인가요?”처럼 추가 정보를 묻고, 사용자가 답하면 최종 예약을 완료합니다. 이런 반복적이고 맥락적인 질의응답 과정을 통해 사용자 요구를 구체화하고 개인화된 결과를 얻을 수 있습니다.

멀티턴 대화에서는 대화 맥락(context) 유지가 무엇보다 중요합니다. 싱글턴 방식에서는 각 질문을 개별적으로 처리하지만, 멀티턴에서는 이전 대화 내용을 기억하면서 이후 질문에 답해야 자연스러운 흐름이 만들어집니다. 예컨대 사용자가 처음에 “James에게 문자 보내줘.”라고 하고 이어서 “보냈어?”라고 묻는 상황을 생각해봅시다. AI가 이전 맥락을 기억하지 못하면 두 번째 질문 “보냈어?”를 이해하지 못하고 엉뚱한 반응을 할 수도 있습니다. 실제로 단순한 챗봇은 이러한 후속 질문을 맥락 상실로 인해 이해하지 못해, 새로운 명령으로 처리하거나 사용자가 질문을 바꾸도록 요구하기도 합니다. 따라서 여러 턴이 이어지는 긴 대화를 원활히 진행하려면 AI가 대화 내내 필요한 정보를 기억하고 활용해야 합니다. 맥락 유지는 사용자가 매번 같은 내용을 반복하지 않아도 자연스럽게 대화를 이어갈 수 있게 해주므로, 사용자 경험을 향상시키는 핵심 요소입니다.

싱글턴 예시: 한 번의 질문과 한 번의 답변으로 작업 완료. 사용자가 “Hey Siri, set an alarm for 5pm.”라고 요청하면, AI가 바로 알람 설정을 완료하고 대화가 종료됩니다 (한 턴으로 끝남).

멀티턴 예시: 여러 차례의 질의응답으로 이루어진 대화. 처음에 사용자가 “Send a text to James.”라고 명령하면, AI가 “What would you like to say to James?”처럼 추가 질문을 합니다. 사용자가 답변을 주면 (“I’ve just left so I’ll be there in 20.”), AI는 그 정보를 활용해 최종적으로 “Your text to James has been sent.”이라는 응답을 합니다. 이처럼 하나의 목표를 달성하기 위해 대화가 여러 턴 진행되는 형태가 멀티턴 대화입니다.

2. 메시지 배열 구조 이해 (System, User, Assistant 역할)

OpenAI의 ChatGPT 모델(API 기준)은 메시지의 배열(list)로 대화 컨텍스트를 입력받습니다. 이 때 각 메시지에는 세 가지 역할(role) 중 하나가 지정됩니다: system (시스템), user (사용자), assistant (AI 응답)입니다. 대화는 보통 다음과 같은 구조로 이루어집니다:

- **System** 역할: AI의 기본 행동방식이나 역할을 지시하는 시스템 메시지입니다. 대화의 초반에 한 번 주로 사용되며, AI에게 특정 캐릭터나 말투, 제약사항 등을 부여합니다. 예를 들어 시스템 메시지로 "당신은 친절한 고객지원 상담원입니다. 항상 공손하고 정확하게 답변하세요."와 같이 지시할 수 있습니다. 이 시스템 메시지는 대화 전체의 톤과 맥락을 설정하며, 멀티턴 대화 내내 AI의 역할 일관성을 유지하는 데 도움을 줍니다.
- **User** 역할: 사용자가 실제로 AI에게 하는 질문이나 요청입니다. 대화 턴마다 새로운 user 메시지로 사용자의 입력이 추가됩니다. 예를 들어 "안녕하세요, 제 주문을 반품하고 싶어요."라는 사용자의 말이 user 메시지로 들어갑니다.
- **Assistant** 역할: AI(ChatGPT)가 생성한 답변 메시지입니다. user 메시지에 대한 응답으로 assistant 메시지가 추가되며, 모델의 출력이 여기에 담깁니다. 예를 들면 위 사용자의 질문에 대해 "도와드리겠습니다. 주문 번호가 어떻게 되시나요?"라는 답변이 assistant 메시지로 기록됩니다.

이러한 메시지 배열(messages)은 대화의 히스토리를 순서대로 포함합니다. 중요한 점은 멀티턴 대화에서는 매 턴 이전까지의 모든 메시지 기록을 포함해서 모델에 보내야 한다는 것입니다. 모델 자체는 이전 대화를 자동으로 기억하지 못하므로(stateless), 이전 턴의 맥락을 유지하려면 과거 시스템+사용자+assistant 메시지들을 모두 다음 API 호출에 다시 제공해야 합니다. 매 턴 새로운 user 메시지를 추가하고, 그에 대한 assistant의 응답을 배열에 추가하는 방식으로 대화가 누적됩니다.

##실습 3.1. Google Colab 실습

예를 들어 Python API를 사용한다면 대화 진행에 따라 아래와 같이 메시지 배열을 관리합니다:

```
!pip install -U openai

from google.colab import userdata
import os
import openai

# Colab에 저장한 Secret에서 API 키 가져와 환경 변수에 등록
os.environ["OPENAI_API_KEY"] = userdata.get('OPENAI_API_KEY')

client = openai.OpenAI()
```

```

messages = [
    {"role": "system", "content": "당신은 도움주는 고객상담 봇입니다."}
]

# 사용자 첫 질문 추가
messages.append({"role": "user", "content": "안녕하세요, 상품을 반품하고 싶어요."})

# API 호출로 assistant 응답 받기
response = client.chat.completions.create(
    messages=messages,
    model="gpt-4o-mini"
)
answer = response.choices[0].message.content # 모델의 답변 텍스트 추출

# assistant 응답을 메시지 히스토리에 추가
messages.append({"role": "assistant", "content": answer})

from pprint import pprint
pprint(messages)

```

위와 같이 대화 이력 전체(`messages`)를 항상 포함해서 API에 보내면, 모델은 그 안의 모든 맥락과 지시에 따라 일관된 응답을 생성합니다. OpenAI Playground에서도 마찬가지로, 이전까지의 대화 기록이 화면에 남아있는 한 모델은 그 컨텍스트를 활용하여 답을 합니다. Playground의 [View Code](#) 기능을 눌러보면 현재까지 대화가 `system`, `user`, `assistant` 구조로 리스트화되어 API에 전달되는 형식을 확인할 수 있습니다.

요약하면, 멀티턴 대화에서는 매 번 메시지 배열 전체를 업데이트하여 모델에게 이전 대화를 상기시켜야 합니다. 이 구조를 이해하는 것이 컨텍스트가 끊기지 않는 대화를 설계하는 기본 전제입니다.

3. Playground 실습: 다양한 시나리오의 멀티턴 대화 설계

이제 OpenAI Playground에서 **실무형 시나리오**별로 멀티턴 대화를 설계하고 실습해보겠습니다. 수강생들은 처음 멀티턴 프롬프팅을 접하므로, 이해를 돕기 위해 **고객 상담, 교육 튜터링, 일정 조율, 업무 브리핑**의 4가지 예시를 다뤄보겠습니다. 각 시나리오마다 2~3턴씩의 간략한 대화를 만들어 보고, 시스템 프롬프트를 활용해 **역할을 설정**한 뒤 **대화 흐름**이 자연스럽게 이어지는 것을 확인해보세요.

예시 1: 고객 상담 (반품 문의) 시나리오

- **설정:** 고객지원 챗봇으로서 친절하고 정확하게 응대하기 (system 메시지 활용).
- **상황:** 사용자가 제품 반품을 문의하고, 챗봇이 절차를 안내하는 대화.

##실습 3.2. OpenAI Playground 실습

시스템: 당신은 온라인 쇼핑몰의 고객지원 담당 챗봇입니다. 고객의 반품/환불 문의를 친절하게 도와주세요.

사용자: 안녕하세요, 지난주에 산 제품을 반품하고 싶어요. 어떻게 하면 되나요?

어시스턴트: 안녕하세요! 이용에 불편을 드려 죄송합니다. 주문 번호를 알려주시면 반품 절차를 도와드리겠습니다.

사용자: 주문 번호는 12345이고, 제품에 결함이 있어요.

어시스턴트: 확인 감사합니다. 주문 번호 12345번 상품의 반품을 접수 하였습니다. 곧 이메일로 반품 송장과 안내를 보내드릴게요. 혹시 추가로 도와드릴 사항이 있을까요?

위 대화에서는 첫 system 메시지로 챗봇에게 “고객지원 담당” 역할과 친절한 어조를 지정했습니다. 그 결과 assistant가 **역할에 맞게 공손한 인사와 안내**로 응답하고, 사용자의 추가 정보 제공에도 맥락을 유지하여 반품 절차를 이어서 설명합니다. 사용자와 두어 차례 주고받는 사이에 **이전 턴의 정보(주문 번호 12345 등)**를 기억하여 최종 답변에 활용하고 있음을 볼 수 있습니다.

예시 2: 교육 튜터링 시나리오

- **설정:** 학생 질문에 단계적으로 답하고, 필요하면 추가 설명을 하는 **과외 튜터** (system 메시지 활용).
- **상황:** 학생이 개념을 질문하면 튜터 AI가 답변하고, 학생이 다시 이해 안 되는 부분을 물어보는 대화.

시스템: 당신은 물리를 가르치는 친절한 과외 교사입니다. 학생의 수준에 맞춰 자세하고 쉽게 설명하세요.

사용자: 선생님, 뉴턴의 두 번째 법칙이 뭐예요?

어시스턴트: 좋은 질문이에요! 뉴턴의 두 번째 운동 법칙은 물체에 작용하는 힘이 가속도와 질량의 곱과 같다는 내용입니다 (공식: $F = m \cdot a$). 간단히 말해 힘을 많이 주면 더 빨리 가속되고, 같은 힘이라도 무거운 물체는 가속도가 작아져요. 이해되나요?

사용자: 음... 그러면 떨어지는 물체에도 이 법칙이 적용되나요?

어시스턴트: 네, 적용됩니다. 예를 들어, 물체가 자유 낙하할 때 지구의 중력이라는 일정한 힘이 작용하죠. 이 힘(F)이 일정하고 물체의 질량(m)에 비례하여 가속도(a)가 결정됩니다. 그래서 무거운 물체일수록 같은 중력에서 가속도는 동일하게 (중력가속도 9.8m/s^2) 유지돼요. 조금 어렵게 느껴지나요? 더 설명이 필요하면 알려주세요!

이 예시에서는 AI를 **물리 과목 튜터**로 설정했습니다. 첫 질문에 대한 답변에서 assistant는 공식과 함께 쉬운 설명을 덧붙였고, 이어서 학생이 후속 질문을 하자 **앞서 설명한 맥락 (F=ma 법칙)**을 바탕으로 새로운 예를 들어 답변합니다. **대화 맥락이 유지**되므로 두 번째 답변에서도 학생이 이미 알고 있는 내용을 반복하지 않고, 이전 답변을 토대로 추가 설명을 제공합니다. 또한 assistant는 학생의 이해도를 확인하며 **교사의 역할**을 일관되게 이어갑니다.

예시 3: 일정 조율 시나리오

- **설정:** 비서 또는 일정 관리 도우미 역할 (system 메시지 활용).
- **상황:** 사용자가 약속 일정을 변경 요청하고, 비서 AI가 변경된 내용을 확인해주는 대화.

시스템: 당신은 사용자의 일정을 관리해주는 비서입니다. 정중하고 효율적으로 일정 변경을 도와주세요.

사용자: 제가 원래 다음 주 월요일 오후 3시에 회의가 있는데, 화요일 같은 시간으로 변경할 수 있을까요?

어시스턴트: 확인해보겠습니다. 말씀하신 회의를 **다음 주 화요일 오후 3시**로 변경했습니다. 새 일정이 맞는지 확인 부탁드립니다. 필요하면 참석자들에게 변경 사항을 공지할게요.

사용자: 네, 화요일 3시면 됩니다. 공지 부탁드립니다. 감사합니다.

어시스턴트: 알겠습니다. 일정이 성공적으로 업데이트되었으며, 관련된 모든 참석자에게 변경된 일정을 공지했습니다. 다른 도움이 필요하시면 언제든지 말씀주세요!

여기서는 AI 비서가 사용자의 일정 변경 요청에 따라 **기존 일정 정보를 기억**하고 새로운 일정으로 바꾸어 응답합니다. Assistant의 첫 답변에서 **사용자의 요청 내용("다음 주 화요일 오후 3시")을 그대로 반영**하여 확인을 구하고 있습니다. 이처럼 이전 user 메시지의 세부사항도 잊지 않고 응답에 포함함으로써 대화의 정확성과 연속성을 보여줍니다. 사용자가 확인을 요청하자, assistant는 맥락을 유지한 채 일정 변경 완료와 공지 사항까지 마무리하여 **역할을 충실히 수행**합니다.

예시 4: 업무 브리핑 시나리오

- **설정:** 직장 상사 혹은 동료에게 브리핑/보고하고 피드백 주고받는 상황 (system 메시지로 역할 설정).
- **상황:** 팀원이 간단한 프로젝트 경과를 보고하고, 상사가 추가 질문이나 피드백을 하는 대화.

시스템: 당신은 사용자의 직장에서의 팀장입니다. 팀원 보고에 대해 간단히 피드백하고 추가 질문을 하세요.

사용자: 팀장님, 이번 주 프로젝트 진행 상황을 보고드리겠습니다. A 작업은 80% 완료되었고, B 작업에서 문제가 생겨 일정을 이틀 지연시켰습니다.

어시스턴트: 보고 감사합니다. A 작업은 거의 완료되어 가는군요. B 작업에서 문제가 있었다고 하셨는데, 현재 그 문제를 해결하기 위해 어떤 조치를 취하고 있나요?

사용자: 개발팀과 논의해서 B 작업 이슈를 해결 중이며, 내일까지 대안을 마련할 예정입니다.

어시스턴트: 알겠습니다. 차주까지 B 작업 정상화에 집중해 주세요. 혹시 지원이 필요하면 알려주시고, 진행 상황을 공유 바랍니다. 수고했어 요!

이 시나리오에서는 AI가 **팀장(상사)** 역할로서 보고를 받는 톤과 말투를 취합니다. Assistant는 사용자의 보고 내용(A작업 80% 완료, B작업 지연)을 **정확히 짚어** 피드백하고, B 작업 문제에 대해 구체적인 추가 질문을 했습니다. 사용자 추가 답변 후에도 그 내용을 반영하여 향후 지시와 격려로 마무리하는 등, **대화 내내 팀장으로서의 일관된 태도**를 유지하고 있습니다. 시스템 메시지를 통해 정한 역할에 따라 assistant의 말투도 격식을 갖추고 핵심을 묻는 식으로 유지되어, 멀티턴 대화 동안 **역할과 컨텍스트의 일관성**이 잘 드러납니다.

위의 실습 예시들을 Playground에서 직접 실행해 보세요. 각 시나리오마다 시스템 프롬프트 설정 → 사용자 입력 → AI 응답 → (필요 시) 추가 사용자 입력 → AI 응답 순으로 2~3턴씩 대화를 진행하면서, **모델이 맥락을 얼마나 잘 유지하는지, 역할에 맞는 톤을 계속 유지하는지** 살펴보는 것이 목표입니다.

4. 컨텍스트 유지 전략 (멀티턴 대화 프롬프트 기법)

멀티턴 대화를 설계할 때는 AI가 이전 대화 맥락을 잃지 않고, 처음 의도한 역할과 톤을 지속적으로 유지하도록 하는 여러 가지 전략을 사용할 수 있습니다. 초심자 분들은 다음의 핵심 기법들을 염두에 두고 프롬프트를 구성하면 좋습니다:

- **시스템 메시지로 역할 고정:** 대화 시작 시 적절한 system 지시를 활용하면, AI가 대화 내내 해당 역할을 일관되게 연기하도록 유도할 수 있습니다. 예를 들어 “당신은 ~~한 어시스턴트다”와 같이 처음에 명확히 규정하면 모델이 중간에 태도를 벗어나지 않습니다. 시스템 프롬프트를 항상 포함하고 유지함으로써 모델이 어떻게 행동해야 하는지를 계속 인지하게 됩니다. 이는 모호성을 줄이고 대화 중간에 불필요한 말투 변화나 역할 이탈을 막아 응답의 일관성과 안정성을 높여줍니다.
- **요약형 응답 활용 (맥락 재확인):** 대화가 진행되면서 AI가 이전 사용자 발언을 짧게 요약하거나 다시 언급하면서 답변하도록 프롬프트를 설계할 수 있습니다. 예를 들어 튜터링 시나리오에서, “~~라는 질문을 주셨는데요, 이에 대한 답은...” 처럼 사용자 질문을 assistant가 먼저 요약하고 답하는 형태입니다. 이렇게 하면 AI가 사용자의 의도를 정확히 파악했는지 확인해 주고, 동시에 요약 내용을 대화 히스토리에 남겨 이후 턴의 맥락 단서로 사용할 수 있습니다. 이러한 기법은 멀티턴 대화에서 문맥 혼동을 줄이고 대화를 부드럽게 이어가는 데 유용합니다.
- **이전 대화 요약을 다음 요청에 삽입:** 대화가 길어질수록 모든 히스토리를 계속 추가하면 토큰 한도에 걸릴 수 있습니다. 이때는 이전 대화 내용을 요약해 압축된 형태로 이어가는 전략을 씁니다. 예를 들어, 10턴째 대화를 진행 중이라면 처음 5턴분의 핵심을 한두 문장으로 정리해 새로운 system 메시지나 user 메시지에 포함하고, 상세한 옛 기록은 생략하는 식입니다. 요약본을 넣을 때는 중요한 고유명사나 결정 사항 등 반드시 기억되어야 할 정보를 담아야 합니다. 이렇게 컨텍스트를 압축하면 모델이 큰 그림의 맥락은 유지하되 불필요한 세부사항으로 인한 토큰 낭비를 줄일 수 있습니다. **Azure OpenAI** 지침에서도 대화 기록이 길어질 경우 과거 부분은 잘라내거나 요약해 넣되, 시스템 메시지와

핵심 맥락은 남겨두라고 권장합니다. 요약 삽입은 특히 토큰 한계로 이전 메시지 일부를 포함하기 어려울 때 매우 유용한 방법입니다. (참고: OpenAI GPT-4 모델은 매우 긴 문맥도 처리할 수 있지만, 여전히 상한이 있으며, 불필요하게 긴 히스토리는 비용과 응답 시간을 늘립니다.)

이러한 전략들을 상황에 맞게 조합하면, 멀티턴 대화에서도 AI가 **초반의 의도와 정보들을 잃어버리지 않고** 계속 일관성 있고 관련성 높은 응답을 생성하도록 도울 수 있습니다. 예를 들어, 시스템 프롬프트로 역할을 고정하고 각 턴의 중요한 내용을 응답에 반영하거나 별도 메모로 유지하면, 모델이 수십 턴이 지나도 처음 맥락을 기억하며 대화할 수 있게 됩니다.

5. Playground 결과 비교 및 대화 흐름 검증

멀티턴 대화를 설계한 후에는 **모델의 응답 흐름이 제대로 유지되는지** 확인하는 과정이 필요합니다. Playground에서 각 시나리오를 실행해보았다면, 다음과 같은 점들을 직접 비교하고 관찰해보세요:

- **맥락 유지 여부:** 모델이 사용자의 두 번째, 세 번째 질문에 답할 때 **첫 질문의 내용을 제대로 기억하고 있는지** 확인합니다. 예를 들어 앞서 월드시리즈 예시에서, “2020년 월드시리즈 우승팀이 누구야?” → “어디에서 열렸어?” 라는 연속 질문을 던졌을 때, 두 번째 질문에 모델이 **이전 질문의 대상(월드시리즈)**을 기억하고 답하는지 살펴봅니다. 제대로 멀티턴 설정이 되었다면 “2020년 월드시리즈는 텍사스 알링턴의 글로브라이프 필드에서 열렸습니다.”처럼 앞의 대답을 토대로 정확한 답을 줄 것입니다. 반면, 컨텍스트를 전달하지 않고 두 번째 질문만 보냈다면 모델은 “무엇을 보냈는지 이해하지 못하겠습니다.”처럼 엉뚱한 답을 할 가능성이 높습니다. 다양한 시나리오에서 **이전 대화 내용을 얼마나 충실히 활용하는지** 확인하세요.
- **역할 지속성:** 대화가 여러 턴 진행된 후에도 AI의 말투나 태도가 처음에 지정한 **시스템 역할에 부합**하는지 점검합니다. 예를 들어 고객 상담 시나리오의 첫 응답에서는 공손했는데, 뒤로 갈수록 캐주얼하게 변하거나 다른 스타일로 답변한다면 **역할 일관성이 무너진 것**입니다. 잘 설계되었다면 3턴, 4턴이 지나도 AI는 계속 고객지원 직원답게 존댓말과 정중한 어조를 유지해야 합니다. 튜터링 예시에서는 처음부터 끝까지 친절한 선생님 말투로 설명하고 있는지 확인하세요. 만약 특정 턴에서 모델이 역할을 벗어나는 듯한 응답을 했다면, **시스템 메시지를 강화하거나 이전 맥락을 다시 상기**시키는 조치를 취해야 합니다.
- **추가 실험:** 필요하다면 **모델의 종류를 바꿔 비교**해보는 것도 좋습니다. 예를 들어 기본으로 사용한 **GPT-4o-mini** 대신 더 성능이 좋은 **GPT-4o** 모델로 동일한 대화를 시도해 보세요. 보다 **맥락 추론 능력이 뛰어난 모델**일수록 작은 프롬프트 변경에도 응답 흐름을 잘 유지할 수 있습니다. 반면, 용량이 작은 모델은 맥락을 잊거나 역할을 일관되게 유지하는데 어려움을 겪을 수 있습니다. 이런 차이를 관찰하면 왜 복잡한 멀티턴 업무에 고성능 모델이 권장되는지 이해할 수 있습니다.

Playground에서는 대화 내용을 수정하거나 초기화하기도 쉽기 때문에, **여러 가지 프롬프트 변형과 시나리오 연장**을 시도해보세요. 혹시라도 어느 지점에서 모델이 엉뚱한 답을 하거나 맥락을 놓쳤다면, 그 부분에서 **컨텍스트 전달이 누락되었거나 모호한 지시**가 있었는지 뒤돌아보고 프롬프트를 개선하면 됩니다.

결과적으로, 멀티턴 대화 실습의 핵심은 **“모델이 대화를 기억하고 있는가? 역할을 잘 수행하고 있는가?”**를 지속적으로 검증하며 프롬프트를 세밀하게 다듬는 것입니다.

6. 실습 정리 및 유의사항 (멀티턴 프롬프트 설계 팁)

2일차 3교시 실습을 통해 멀티턴 대화 프롬프트의 개념과 구현을 경험해봤습니다. 마지막으로, 멀티턴 설계를 할 때 기억해야 할 몇 가지 유의점과 추가 전략을 정리하겠습니다:

- **대화 히스토리 관리:** 멀티턴에서는 항상 지금까지의 **대화 내용을 빠짐없이 유지**해야 합니다. 새로운 사용자 입력을 처리할 때 이전 메시지를 빼먹으면 모델이 맥락을 잃어버리니, `messages` 배열 업데이트를 철저히 하는 것이 중요합니다. 모델은 과거 대화를 스스로 기억하지 못하므로, 개발자가 대화 내역을 잘 관리해야 합니다.
- **컨텍스트 초과 방지:** 대화가 길어져 메시지가 매우 많아지면 모델 입력 토큰 한도에 부딪힐 수 있습니다. 예를 들어 GPT-4 모델은 수만 개의 토큰까지도 처리하지만, 무제한은 아닙니다. 따라서 **너무 오래된 대화는 요약하거나 제거**해서 토큰을 절약해야 합니다. 핵심 정보는 잃지 않도록 요약하고, 부수적인 부분은 생략하는 식으로 조절합니다. 이렇게 하면 토큰 한도를 초과하지 않으면서도 모델이 일관성을 유지하도록 할 수 있습니다. (필요하다면 일정 길이마다 대화 내용을 정리하여 system 메시지로 재주입하는 방법도 있습니다.)
- **명확한 지시와 대화 힌트:** 멀티턴 프롬프트에서는 **모델에게 기대하는 바를 구체적으로 알려주는 것**이 특히 중요합니다. 대화 도중에 새로운 목표가 생기거나 톤을 바꿔야 할 경우, 그때그때 **시스템 메시지나 사용자 메시지에 힌트**를 추가하여 모델을 안내할 수 있습니다. 예를 들어 중간에 **요약**이 필요하다면 사용자 메시지로 **“방금 대화 내용을 한 문장으로 요약해줘.”**라고 명시적으로 요구할 수도 있고, 또는 시스템 메시지로 **“assistant는 사용자의 마지막 발언을 간략히 정리하고 답한다.”**는 지침을 추가할 수도 있습니다. 이렇게 프롬프트 자체에 원하는 대화 흐름에 대한 **힌트나 가이드라인**을 넣으면 모델이 방향을 잃지 않고 대화를 이어가는 데 도움이 됩니다.
- **일관성 및 품질 모니터링:** 멀티턴 대화 중 모델이 **일관성을 잃거나 앞뒤가 맞지 않는 답변**을 하는 순간이 있는지 항상 모니터링하세요. 예를 들어 동일한 사실을 두 번 물었는데 답이 다르게 나온다면, 이전 답변을 뒤집는 내용을 말한다면 하는 경우입니다. 이런 문제가 발생하면 대화 기록을 검토하여 필요한 맥락이 누락되지 않았는지 확인하고, 필요하다면 시스템 메시지로 사실관계를 다시 상기시키거나 모델의 설정을 조정합니다. 멀티턴에서는 작은 혼동이 누적되면 크게 일탈할 수 있으므로 **초기에 바로잡는 것**이 중요합니다.

- **테스트와 개선 반복:** 멀티턴 프롬프트는 한 번에 완벽하게 만들기 어렵습니다. **여러 시나리오를 테스트**하면서 프롬프트를 조금씩 개선해 나가세요. 예를 들어, 고객상담 시나리오에서 반품이 아닌 교환 요청으로 질문을 바꿔보거나, 튜터링 시나리오에서 학생이 갑자기 다른 챕터를 질문하도록 해보는 등 변주를 쥘보는 겁니다. 모델이 새 상황에서도 맥락을 유지하고 적절히 대응하면 프롬프트가 견고한 것이고, 헤매기 시작하면 그 지점에서 어떤 **추가 지시나 맥락 보강**이 필요한지 생각해볼 수 있습니다.

마지막으로, 멀티턴 대화를 다룰 때는 **사람과의 대화처럼 유연하고 논리적인 흐름**을 목표로 삼으세요. 초기 시스템 프롬프트로 큰 가이드라인을 주고, 각 턴의 상호작용에서 **사용자의 의도를 정확히 파악**하며, 필요하면 **요약과 재확인**을 통해 오해를 줄이는 것이 핵심입니다. 이런 원칙을 지킨다면 고객 상담, 교육, 일정 관리, 업무 보고 등 어떤 실제 상황에서도 유용한 멀티턴 AI 프롬프트를 만들 수 있을 것입니다.